

Abstract

The Disaster Alert & Community Response App is a centralized platform designed to improve communication and coordination during disasters such as floods, cyclones, earthquakes, fires, and other emergencies. The application allows users to receive location-based alerts, report emergency incidents, send SOS requests, and access nearby shelters and emergency services. It also provides a live incident map and priority-based help requests to support faster response. The system uses a web-based frontend with a backend API and MongoDB database for managing users, alerts, incidents and response information. The main goal is to provide a centralized platform for faster reporting, better coordination, and effective community response during emergencies.

Tools & Technologies

- Frontend: HTML, CSS, JavaScript
- Backend: Node.js, FastAPI
- Database: MongoDB
- API: REST API
- Mapping: Leaflet / OpenStreetMap
- Development Tool: VS Code
- Version Control: Git / GitHub



Problem Statement & Objectives

Problem Statement

During disasters, people may face delays in receiving relevant alerts, reporting emergencies, requesting help, and locating nearby emergency services. Existing approaches may lack centralized communication and coordination between affected users, responders, and the community.

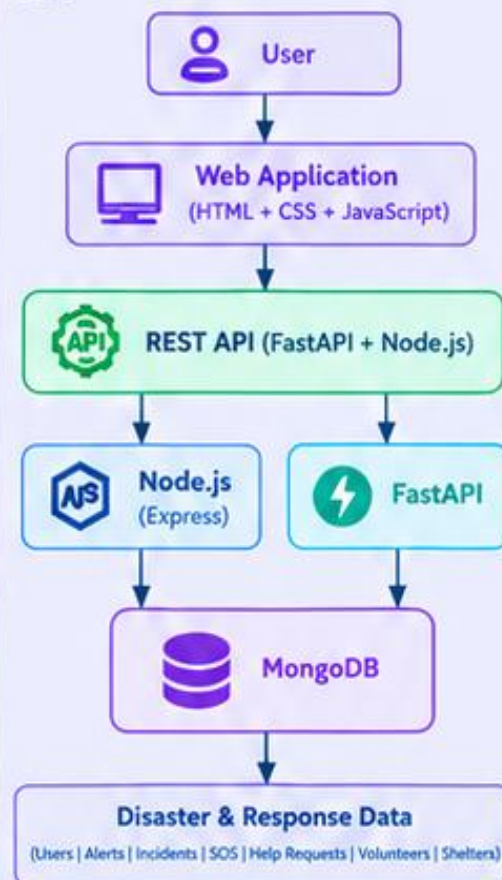
Objectives

- Provide quick SOS-based emergency reporting.
- Provide location-based disaster alerts and a live incident map.
- Enable priority-based emergency help requests.
- Improve coordination between users, responders and community volunteers.
- Provide access to nearby shelters, hospitals and emergency services.

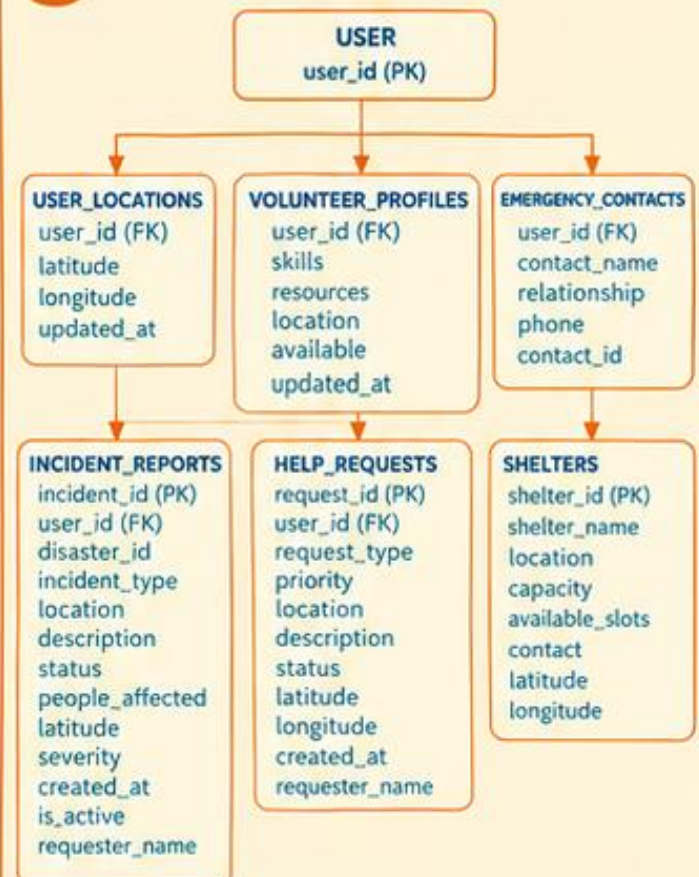
Scope

- Covers user authentication, disaster alerts, SOS reporting, live mapping, help requests, family safety and community assistance.

System Architecture



ER Diagram & Database Design



Database: MongoDB (safesphere)

Purpose: Stores user locations, volunteer profiles, shelters, incident reports, emergency help requests and emergency contacts to support disaster monitoring and community response.

Existing vs Proposed System

Feature	Existing System	Proposed System
Disaster Alerts	[General / non-centralized]	[Location-based]
Emergency Reporting	[Normal reporting]	[One-click SOS reporting]
Incident Location	[Limited information]	[Live map with markers]
Help Requests	[General requests]	[Priority-based (Critical/High/Medium/Low)]
Emergency Services	[Hard to locate]	[Nearby shelters, hospitals & services]
Community Support	[Limited coordination]	[Volunteer & community help]
Family Safety	[Limited check-in]	["I'm Safe" check-in]
Verification	[Reports may lack verification]	[Verified alerts & report status]

Feasibility Study

- Technical:** Uses commonly available web technologies (HTML, CSS, JavaScript, Node.js, FastAPI, MongoDB).
- Economic:** Widely available tools, reducing development and deployment costs.
- Operational:** Simple dashboard with sections for alerts, SOS, map, help requests, family safety and services.
- Schedule:** Can be developed incrementally (authentication, alerts, SOS, mapping, help requests, community response).

Implementation Screenshots



Results & Testing

Results

- Developed a centralized disaster-response web application.
- Implemented user dashboard and SOS functionality.
- Added disaster alerts and live incident mapping.
- Implemented priority-based help requests.
- Provided access to shelters and emergency services.
- Added community and family-safety features.

Testing

- Login:** User authentication tested.
- SOS:** Emergency categories and submission tested.
- Alerts:** Disaster alert information displayed.
- Live Map:** Disaster and service markers displayed.
- Help Requests:** Critical, High, Medium, Low priorities.
- Navigation:** Dashboard modules accessible.

Conclusion

The Disaster Alert & Community Response App provides a centralized platform for managing emergency information and community response. The application combines SOS reporting, location-based alerts, live incident mapping, priority-based help requests and emergency service information to support faster communication and coordination during disasters.

Future Scope

- Real-time push notifications & SMS alerts
- AI-based disaster detection & risk prediction
- Automated emergency-service integration
- Advanced volunteer-response matching
- Real-time evacuation route recommendations
- Fake-report and duplicate-report detection
- Integration with official authorities

References:

- [1] MongoDB Documentation [2] FastAPI Documentation [3] Node.js Documentation [4] Leaflet Documentation [5] OpenStreetMap Documentation [6] GitHub Documentation

QR: GitHub



<https://github.com/tejasri1473/Disaster-Alert-AND-Community-Response>